Revue des trois visuels

Projet Z-CORE / SAHBI-SA V6.1 — NetSecurePro IA · 1er octobre 2026

Conclusion : ces visuels ne sont pas publiables en l'état. Ils contiennent des fautes dans le texte et dans les URL, et leurs chiffres se contredisent.

1. Incohérences chiffrées

Source	Valeur affichée
Visuel 1	« 8 ms Local » et « 0.002ms »
Kit (README, publications)	« 0.0009 ms »
Cible citée	« < 1.2 ms »

Trois latences différentes pour un même système sont inacceptables dans une publication technique. Il faut retenir une seule mesure, accompagnée de sa méthode (médiane, p99, nombre d'itérations, périmètre : appel en processus ou non), et la reporter partout. Sans cela, un lecteur technique remettra en cause l'ensemble.

2. Visuels 1 et 3 (affiches « SAHBI SA »)

Fautes de texte

- « discuver » (visuel 1) et « dissover » (visuel 3) pour « découvrir ».
- « mon Génie et Réal » est inintelligible pour un public externe. Reformulez en une promesse claire.

URL imprimées

- **Visuel 1 :** `milyesgithub-io` (point manquant) et `milyeys.github-o.io//Bin-Lancer V6-Puce.html` (« milyeys », « github-o.io », double barre, espace dans le chemin).
- **Visuel 3 :** `milyeus.github-io*/ SAHHVbs-SA8/Lancer-V6-Puce.html` (« milyeus », astérisque, espace, « SAHHVbs-SA8 »).
- Les deux affiches donnent des URL différentes pour la même ressource, et aucune n'est valide : le domaine correct est `milyes.github.io`.

QR codes

Leur contenu ne peut pas être vérifié depuis une capture. Si le texte imprimé est faux, rien ne garantit que le code est correct. Scannez-les et comparez avec l'URL réelle. Régénérez-les à partir d'une URL canonique unique, en gardant un contraste élevé et une zone de silence. Le logo central du visuel 3 réduit la tolérance aux erreurs : utilisez le niveau de correction H ou retirez-le.

3. Visuel 2 (schéma d'architecture)

Il est clairement généré par IA, avec du texte altéré :

- **Libellés illisibles ou inventés :** « snarend-umiong packet », « nlood packets », « gore-x packets », « branch --vnl ».
- **Chaîne de licence corrompue :** `org.mestring/ore9mt&IAoEMEDL2BBstkrgR-`. Un sceau de licence illisible nuit à la crédibilité et évoque une certification qui n'existe pas.
- `z_core_version: 2.0.1` alors que tout le reste parle de V6.1.
- `integration_target: MILYES-IA-V9-NANS-CORE` : nom absent du kit.
- `zcore-api/claude-protocol-bridge` alors que le kit parle de `zpuce-protocol-bridge`. Deux désaccords : le nom, et le fait que « Claude » dans un composant puisse laisser croire à un lien avec Anthropic. Nommez le composant `zpuce-protocol-bridge` partout.
- Le schéma décrit mal les modules du kit (`INFERENCE_CORE`, `BUS_DIRECT`, `PREUVE_LOI25`, `ZERO_TRUST_GATE`).

Pour une publication visant la crédibilité technique, remplacez-le par un diagramme vectoriel exact (SVG ou Mermaid), généré à partir de la liste réelle des modules.

4. Défauts de format

- Ce sont des captures d'écran de téléphone avec bandes noires et recadrage approximatif. Exportez depuis les fichiers sources, au format carré 1080×1080.
- La page 3 du carrousel prévue (console Termux avec la latence) est absente. Elle ne doit être produite qu'avec une vraie exécution de `generer_kit.py` et de la mesure branchée sur le bus.
- La page 4 (extrait du JSON d'audit) doit provenir du fichier généré par le script corrigé, avec les hachages SHA-256 complets.

5. Ordre d'exécution recommandé

1. Fixer une URL canonique et la tester dans un navigateur.
2. Mesurer la latence réellement (script corrigé, `PING` branché) et reporter une seule valeur.
3. Régénérer les QR codes, puis les scanner.
4. Corriger les textes des affiches (fautes, formulation).
5. Remplacer le schéma IA par un diagramme exact.
6. Produire les captures de console et du JSON depuis cette exécution.

Suite proposée

Production du diagramme d'architecture corrigé en SVG (couleurs Z-CORE, 1080×1080), ainsi que d'un script stdlib de vérification des URL et de génération de QR si une dépendance est acceptable. Prérequis : l'URL canonique à retenir.